



Computational Logic in the Era of AI

Simply Typed Lambda Calculus

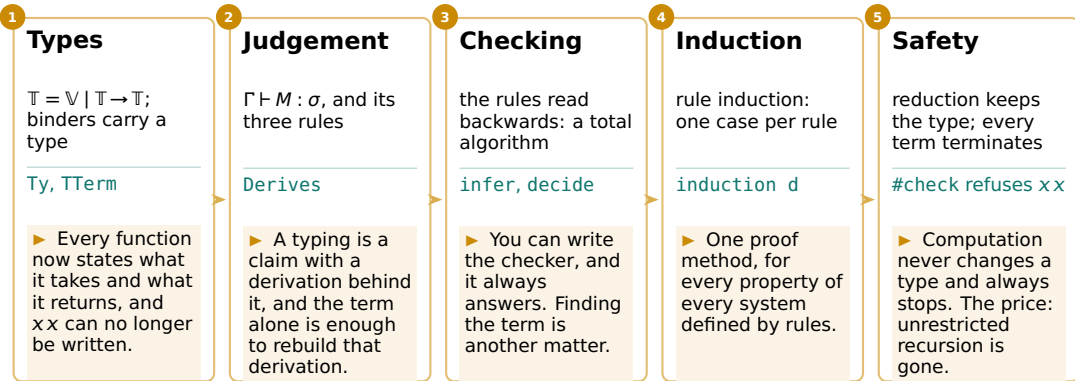
Sergey Mechtaev
mechtaev@pku.edu.cn

Term 1 • 26–27

This lecture

↓ you are here

■ theory ■ in Lean



History: types against self-reference



**Bertrand
Russell**

1872–1970

Types enter logic,
to stop a set from
containing itself.

- 1901** Russell's paradox: the set of all sets that are not members of themselves.
- 1908** Russell's theory of types: a set may only contain objects of the type below its own.
- 1935** Kleene and Rosser: the untyped λ -calculus, taken as a logic, is inconsistent; the source is self-application.
- 1940** Church, *A formulation of the simple theory of types*: λ -terms with types written on the binders. This lecture's system.
- 1969** Hindley, then Milner (1978): leave the types off and let the compiler find them. Typing à la Curry: ML, Haskell.

Russell's paradox and the Kleene–Rosser inconsistency have the same source: an object applied to, or containing, itself. Types exclude both by construction. Photos: Wikimedia.



Alonzo Church

1903–1995

Types enter the
 λ -calculus, to stop
a function from
being applied to
itself.

N&G §2.14

Simple types

The system of this lecture is $\lambda \rightarrow$, the simply typed λ -calculus. Its types: start from an infinite set of type variables $\mathbb{V} = \{\alpha, \beta, \gamma, \dots\}$ and close under one rule.

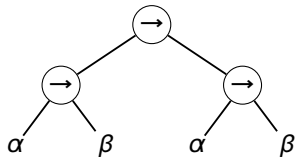
Definition (simple types, $\mathbb{T}$)

1. (Type variable) If $\alpha \in \mathbb{V}$ then $\alpha \in \mathbb{T}$;
2. (Arrow type) If $\sigma, \tau \in \mathbb{T}$ then $(\sigma \rightarrow \tau) \in \mathbb{T}$.

In abstract syntax: $\mathbb{T} = \mathbb{V} \mid \mathbb{T} \rightarrow \mathbb{T}$.

Nothing else is a type: $\mathbb{T}$ is the smallest such set.

Read a type variable as a basic type such as `Nat` or `String`, and an arrow type as a function type such as `Nat  $\rightarrow$  String`.
N&G §2.2, Def. 2.2.1



$(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$

a type is a tree, as a term is

Two associativities, pointing opposite ways

The arrow is right-associative

$$\rho \rightarrow \sigma \rightarrow \tau \equiv \rho \rightarrow (\sigma \rightarrow \tau)$$

Application is left-associative

$$f x y \equiv (f x) y$$

The two fit because each argument consumes the first arrow of the type: the term gains an argument on the right, the type loses an arrow on the left.

$$f : \rho \rightarrow \sigma \rightarrow \tau$$

$$f x : \sigma \rightarrow \tau \quad x : \rho \text{ consumes } \rho \rightarrow$$

$$f x y : \tau \quad y : \sigma \text{ consumes } \sigma \rightarrow$$

the term grows

the type shrinks

So $\alpha \rightarrow \beta \rightarrow \gamma$ takes two arguments; $(\alpha \rightarrow \beta) \rightarrow \gamma$ takes one, a function. Those parentheses cannot be dropped.

N&G Notation 2.2.2 (4), 1.3.10, Rem. 2.2.8

$\mathbb{T}$, as an inductive

Two clauses, two constructors; infixr makes the arrow right-associative.

```
inductive Ty where
  | base  : String → Ty
  | arrow : Ty → Ty → Ty

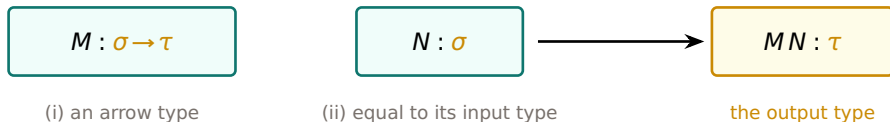
infixr:25 " ⇒ " => Ty.arrow      -- infixr: right-associative

#check (Ty.base "α" ⇒ Ty.base "β" ⇒ Ty.base "α")
-- α ⇒ (β ⇒ α)
```

Typing an application, and typing an abstraction

Application: two conditions

If $M : \sigma \rightarrow \tau$ and $N : \sigma$, then $MN : \tau$. So MN has a type only if (i) the type of M is an arrow type $\sigma \rightarrow \tau$, and (ii) the type of N is σ , the input type of that arrow.



Abstraction: no condition

If $x : \sigma$ and $M : \tau$, then $\lambda x:\sigma. M : \sigma \rightarrow \tau$. Abstracting a typed term always yields a typed term.

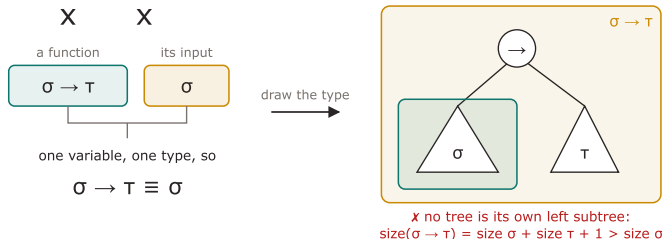
xx has no type

Proposition

There are no types σ, τ with $x : \sigma$ and $xx : \tau$.

N&G Ex. 2.2.6 (3)

Proof. Suppose $xx : \tau$. By condition (i) the first x has an arrow type, $x : \sigma \rightarrow \tau$; by condition (ii) the second has its input type, $x : \sigma$. A variable has one type, so $\sigma \rightarrow \tau \equiv \sigma$: a tree equal to its own proper subtree, which no finite tree is. $\square$



Two consequences

1. Ω is not a $\lambda \rightarrow$ term

$\Omega \equiv \omega \omega$ with $\omega \equiv \lambda x. x x$. Whatever type is written on the binder, $\lambda x:\sigma. x x$ has a body $x x$ with no type, so ω has none and Ω cannot be formed. The term that reduced to itself forever is gone.

2. Y is not a $\lambda \rightarrow$ term

$Y \equiv \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$ contains $x x$ twice, so it cannot be given a type either. The fixed-point combinator, and with it recursion by self-application, is gone.

N&G §2.12

The size argument, as a tactic proof

```
def size : Ty → Nat
| .base _      => 1
| .arrow a b   => size a + size b + 1

theorem arrow_ne (σ τ : Ty) : (σ ⇒ τ) ≠ σ := by
  intro h
  have := congrArg size h
  simp [size] at this
  omega
```

step	what it does	state afterwards
intro h	assume it ($\neq$ is “ $\dots \rightarrow \text{False}$ ”)	$h : \sigma \Rightarrow \tau = \sigma \quad \vdash \text{False}$
have := congrArg size h	apply size to both sides	$\text{this} : \text{size}(\sigma \Rightarrow \tau) = \text{size } \sigma$
simp [size] at this	unfold size on the left	$\text{this} : \text{size } \sigma + \text{size } \tau + 1 = \text{size } \sigma$
omega	no solution in $\mathbb{N}$, hence False	closed

Ill-typed programs are rejected

Condition (ii), checked on every application: the argument's type must equal the input type.

```
def double (n : Nat) : Nat := 2 * n
```

```
/-- error: Application type mismatch: The argument  
    "one"
```

```
has type
```

```
    String
```

```
but is expected to have type
```

```
    Nat
```

```
in the application
```

```
    double "one" -/
```

```
#guard_msgs in
```

```
-- records the error Lean reports
```

```
def bad : Nat := double "one"
```

Two ways to type a variable

À la Church: explicit

Every binder carries its type:

$\lambda z:\beta. \lambda u:\gamma. z$

The type of a term is then computed.
Uniqueness holds.

Church typing, throughout

In mathematics and logic the types of variables are given beforehand, so we take them as given and never search for them. Lean does the same underneath, as the next slide shows.

À la Curry: implicit

Binders are bare; types are searched for, with guesses:

$\lambda z. \lambda u. z$

Every $\lambda z. \lambda u. z$ has type $A \rightarrow C \rightarrow A$,
for any A, C . Uniqueness fails.

N&G §2.3

Lean is Church-typed underneath

I How a missing binder type is filled in

Before checking, a binder without a type gets a placeholder `?m`, which is solved from the expected type of the term. An unsolved placeholder is an error, so the checker sees only annotated binders.

```
-- 1. Annotated binder: reaches the checker as written.
#check fun (x : Nat) => x           -- fun (x : Nat) => x : Nat → Nat
-- 2. Bare binder, expected type Nat → Nat: Nat is written on it.
#check (fun x => x : Nat → Nat)    -- fun (x : Nat) => x : Nat → Nat
-- 3. Bare binder, no expected type: the placeholder ?m.1 stays,
--    and a definition may not contain one.
#check fun x => x                  -- fun (x : ?m.1) => x : ?m.1 → ?m.1
-- rejected: "Failed to infer type of binder 'x'"
#guard_msgs (drop error) in
def idc := fun x => x
```

Pre-typed terms Λ_T

Λ is revised so that the binder records a type:

$$\Lambda_T = \mathbb{V} \mid (\Lambda_T \Lambda_T) \mid (\lambda \mathbb{V}:\mathbb{T}. \Lambda_T)$$

-- Def. 2.4.1. Λ_T : the binder carries its type (typing à la Church, §2.3).

```
inductive TTerm where
  | var : String → TTerm
  | app : TTerm → TTerm → TTerm
  | lam : String → Ty → TTerm → TTerm
```

-- Example 2.4.6's term, $\lambda y:\alpha \Rightarrow \beta. \lambda z:\alpha. y z$, as a value:

```
def ex246 : TTerm :=
  .lam "y" ( $\alpha \Rightarrow \beta$ ) (.lam "z"  $\alpha$  (.app (.var "y") (.var "z")))
```

N&G §2.4, Def. 2.4.1

Statement, declaration, context, judgement

I Definition

- a **statement** is $M : \sigma$ with $M \in \Lambda_{\top}$, $\sigma \in \mathbb{T}$; M is the subject, σ the type;
- a **declaration** is a statement whose subject is a variable;
- a **context** is a list of declarations with different subjects;
- a **judgement** is $\Gamma \vdash M : \sigma$, with Γ a context and $M : \sigma$ a statement.

$$\boxed{x : \alpha \rightarrow \alpha, y : (\alpha \rightarrow \alpha) \rightarrow \beta} \vdash \boxed{(\lambda z : \beta. \lambda u : \gamma. z)(y x)} : \boxed{\gamma \rightarrow \beta}$$

context: the types of
the free variables

the term: teal, in every
diagram of this lecture

its type:
gold

A term M is **legal** if $\Gamma \vdash M : \rho$ for some context Γ and type ρ . Which judgements hold is fixed by the derivation rules, next. N&G Def. 2.4.2, Ex. 2.3.2, Def. 2.4.10

A context is a list

`abbrev Ctx := List (String × Ty)`

`#check ([("x",  $\alpha$ ), ("y",  $\alpha \Rightarrow \beta$ )] : Ctx)`

lookup finds the nearest declaration of a name.

N&G Def. 2.4.2

The derivation rules of $\lambda \rightarrow$

$$\frac{}{\Gamma \vdash x : \sigma} \text{ (var) } \text{ if } x : \sigma \in \Gamma$$

$$\frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau} \text{ (appl)}$$

$$\frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x : \sigma. M : \sigma \rightarrow \tau} \text{ (abst)}$$

One rule per shape of term: variable, application, abstraction. $\Gamma \vdash M : \sigma$ holds exactly when some finite tree of rule instances has it at the root. N&G Def. 2.4.5

The rule (*var*)

$$\frac{}{\Gamma \vdash x : \sigma} \text{ (var) } \text{ if } x : \sigma \in \Gamma$$

A variable has the type its declaration in Γ gives it, and no other. The rule has no premiss, so every derivation begins with instances of it.

■ Instance

$y : \alpha \rightarrow \beta, z : \alpha \vdash y : \alpha \rightarrow \beta$ and $y : \alpha \rightarrow \beta, z : \alpha \vdash z : \alpha$.

The rule (*appl*)

$$\frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau} \text{ (appl)}$$

The two conditions on an application, now under a context: M has an arrow type and N has its input type, both derived in the same Γ . The application then has the output type τ .

■ Instance

From $y : \alpha \rightarrow \beta$, $z : \alpha \vdash y : \alpha \rightarrow \beta$ and $y : \alpha \rightarrow \beta$, $z : \alpha \vdash z : \alpha$:
 $y : \alpha \rightarrow \beta$, $z : \alpha \vdash yz : \beta$.

The rule (*abst*)

$$\frac{\Gamma, x:\sigma \vdash M : \tau}{\Gamma \vdash \lambda x:\sigma. M : \sigma \rightarrow \tau} \text{ (abst)}$$

The premiss types the body M with $x : \sigma$ added to the context. The conclusion removes that declaration from the context and records σ on the binder instead: the context shrinks by exactly the declaration that moves into the term.

■ Instance

From $y : \alpha \rightarrow \beta, z : \alpha \vdash yz : \beta$: $y : \alpha \rightarrow \beta \vdash \lambda z:\alpha. yz : \alpha \rightarrow \beta$.

The three rules, as an inductive

An inductively defined relation: the constructors are the rules, and a value of the type is a derivation.

-- Def. 2.4.5, verbatim: one rule per shape of term.

```
inductive Derives : Ctx → TTerm → Ty → Prop where
| var {Γ x σ} :
  Γ.lookup x = some σ →
  Derives Γ (.var x) σ
  -- (var)
| app {Γ M N σ τ} :
  Derives Γ M (σ ⇒ τ) → Derives Γ N σ →
  Derives Γ (.app M N) τ
  -- (appl)
| abs {Γ x M σ τ} :
  Derives ((x, σ) :: Γ) M τ →
  Derives Γ (.lam x σ M) (σ ⇒ τ)
  -- (abst)
```

notation:40 Γ " ⊢ " M " : " σ => Derives Γ M σ

One derivation, in tree format

One judgement, derived: $\emptyset \vdash \lambda y:\alpha \rightarrow \beta. \lambda z:\alpha. yz : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta.$

(i) $y:\alpha \rightarrow \beta, z:\alpha \vdash y : \alpha \rightarrow \beta$

(ii) $y:\alpha \rightarrow \beta, z:\alpha \vdash z : \alpha$

(appl)

(iii) $y:\alpha \rightarrow \beta, z:\alpha \vdash yz : \beta$

(abst)

(iv) $y:\alpha \rightarrow \beta \vdash \lambda z:\alpha. yz : \alpha \rightarrow \beta$

(abst)

(v) $\emptyset \vdash \lambda y:\alpha \rightarrow \beta. \lambda z:\alpha. yz : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$

Faithful to the rules, but it spreads: the context is copied into every node.

N&G

Ex. 2.4.6, Fig. 2.1

The same derivation, in linear format

Impose an order on the steps and number them, as a book numbers its lines.

(i)	$y:\alpha \rightarrow \beta, z:\alpha \vdash y : \alpha \rightarrow \beta$	(var)
(ii)	$y:\alpha \rightarrow \beta, z:\alpha \vdash z : \alpha$	(var)
(iii)	$y:\alpha \rightarrow \beta, z:\alpha \vdash yz : \beta$	(appl) on (i), (ii)
(iv)	$y:\alpha \rightarrow \beta \vdash \lambda z:\alpha. yz : \alpha \rightarrow \beta$	(abst) on (iii)
(v)	$\emptyset \vdash \lambda y:\alpha \rightarrow \beta. \lambda z:\alpha. yz : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$	(abst) on (iv)

The tree structure is lost, so a column of justifications replaces it. The order is not unique: (i) and (ii) may be swapped. Dependency between judgements is a strict partial order, and any linearisation of it is a valid reading.

Still redundant: the context is written out five times.

N&G §2.5

The same derivation, in flag format

Put each declaration in a flag, and let it govern every line under its pole.

(a)	$y : \alpha \rightarrow \beta$	
(b)	$z : \alpha$	
(1)	$y : \alpha \rightarrow \beta$	(var) on (a)
(2)	$z : \alpha$	(var) on (b)
(3)	$yz : \beta$	(appl) on (1), (2)
(4)	$\lambda z : \alpha. yz : \alpha \rightarrow \beta$	(abst) on (3)
(5)	$\lambda y : \alpha \rightarrow \beta. \lambda z : \alpha. yz : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$	(abst) on (4)

No context is written twice. Closing a flag is the (*abst*) rule, and a flag is the assumption of ($\Rightarrow$ -intro) in natural deduction. N&G §2.5, Ex. 2.4.9, ch. 11

That derivation, as a term

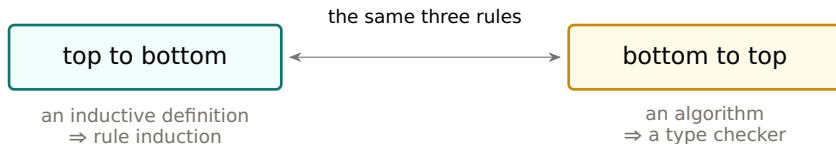
A derivation is a value of type $\Gamma \vdash M : \sigma$, built from the three constructors: `.abs` for (*abst*), `.app` for (*appl*), `.var` for (*var*). The side condition of (*var*), $\Gamma.\text{lookup } x = \text{some } \sigma$, is proved by `rfl`: both sides compute to the same value.

-- Example 2.4.6, as a value: line (5), then (4), (3), (1) and (2)
`example` : [] $\vdash$ ex246 : $((\alpha \Rightarrow \beta) \Rightarrow \alpha \Rightarrow \beta)$:=
 `.abs (.abs (.app (.var rfl) (.var rfl)))`

Rules read in two directions

Top to bottom: from the premisses, derive the conclusion. The rules define, inductively, the set of derivable judgements.

Bottom to top: from the conclusion, find the premisses. Given the term, at most one rule applies, and its premisses are the subproblems.



N&G Rem. 2.4.7

The three problems

One judgement, and three places to put a question mark.

1  $\vdash$ M : 

Well-typedness

is there any context and type for M ?
with Γ given, Type Assignment: compute the type

infer Γ M

2 $\Gamma \vdash M$  σ

Type Checking

does M have type σ in Γ ? decidable:
compute the type, compare it with σ

the checker

3 $\Gamma \vdash$  : σ

Term Finding

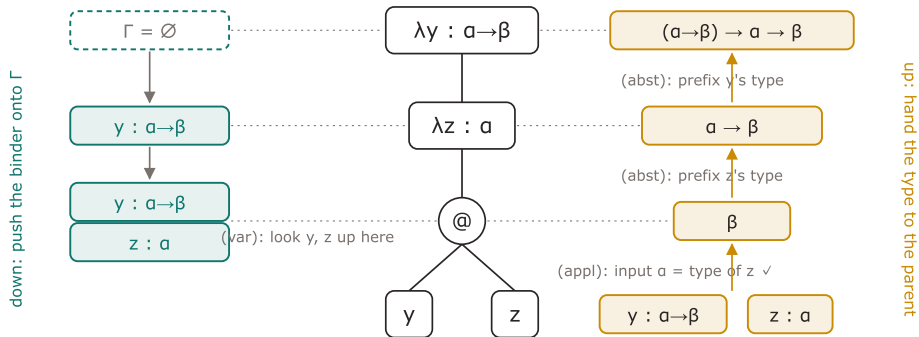
is there a term of type σ at all?
with $\Gamma = \emptyset$: is σ provable?

the prover

All three are decidable in $\lambda \rightarrow$. In the empty context, Term Finding asks whether the type σ , read as a proposition, has a proof. N&G §2.6, §2.9, Thm. 2.10.10

The checker walks the tree

The rules read bottom-up on Example 2.4.6's term: down the tree, each binder is pushed onto the context; back up, each node hands its type on.



One comparison happens, at the application node; everything else is bookkeeping.

The rules, read backwards, are an algorithm

The previous picture, as a def: one clause per node shape.

```
-- Rem. 2.4.7 reads the rules bottom-to-top: each is "a guide to
-- obtain some goal". Read that way, Def. 2.4.5 IS an algorithm.
def infer ( $\Gamma$  : Ctx) : TTerm  $\rightarrow$  Option Ty
| .var x      =>  $\Gamma$ .lookup x
| .app M N    =>
    match infer  $\Gamma$  M, infer  $\Gamma$  N with
    | some ( $\sigma \Rightarrow \tau$ ), some  $\sigma'$  => if  $\sigma = \sigma'$  then some  $\tau$  else none
    | -, -                                     => none
| .lam x  $\sigma$  M => (infer ((x,  $\sigma$ ) ::  $\Gamma$ ) M).map ( $\sigma \Rightarrow \cdot$ )

#eval infer [] ex246                -- some (( $\alpha \Rightarrow \beta$ )  $\Rightarrow$   $\alpha \Rightarrow \beta$ )
#eval infer [] (.app ex246 ex246)   -- none
```

Structural recursion on the term, so it is total: the checker always answers.

Type Checking is decidable

Theorem

For all Γ, M, σ it is decidable whether $\Gamma \vdash M : \sigma$ holds: there is an algorithm that terminates on every input and answers yes exactly when the judgement is derivable.

N&G Thm. 2.10.10 (2)

The algorithm is `infer`: compute `infer  $\Gamma M$` and compare the result with σ . It terminates on every input, by structural recursion on M . That its answers are the right ones is two lemmas:



- **Soundness:** every answer is backed by a derivation. A yes is never wrong.
- **Completeness:** every derivable judgement is answered. So a no means that no derivation exists.

Soundness: infer never answers wrongly

Lemma

If $\text{infer } \Gamma M = \text{some } \sigma$, then $\Gamma \vdash M : \sigma$.

Proof by induction on M , following the three clauses of `infer`. Each clause returns an answer only under the condition written in its rule, so in each case the induction hypotheses are exactly the premisses that rule needs.

M	<code>infer</code> returned some σ because ...	so, by the hypotheses and one rule
x	$\Gamma.\text{lookup } x = \text{some } \sigma$	(<i>var</i>)
PQ	$\text{infer } \Gamma P = \text{some } (\rho \rightarrow \sigma), \text{ infer } \Gamma Q = \text{some } \rho$	$\Gamma \vdash P : \rho \rightarrow \sigma, \Gamma \vdash Q : \rho$, then (<i>appl</i>)
$\lambda x:\rho. P$	$\sigma \equiv \rho \rightarrow \tau$ and $\text{infer } (\Gamma, x:\rho) P = \text{some } \tau$	$\Gamma, x:\rho \vdash P : \tau$, then (<i>abst</i>)

In the middle row the test $\rho = \rho'$ inside `infer` passed, which is what lets (*appl*) fire. □

Soundness, in Lean

Recursion on the term, one clause per row of the table. In the application case the two splits take the match and the if apart; every branch but one contradicts h.

```
theorem infer_sound :  $\forall \{\Gamma M \sigma\}, \text{infer } \Gamma M = \text{some } \sigma \rightarrow \Gamma \vdash M : \sigma$   
  |  $\Gamma, .\text{var } x, \quad \sigma, h \Rightarrow .\text{var } h$  -- h is the side condition  
  |  $\Gamma, .\text{lam } x \rho P, \sigma, h \Rightarrow$  --  $\sigma$  is  $(\rho \Rightarrow \tau)$ , from some  $\tau$   
    obtain  $\langle \tau, hP, rfl \rangle := \text{Option.map\_eq\_some\_iff.mp } h$   
    exact .abs (infer_sound hP)  
  |  $\Gamma, .\text{app } P Q, \quad \sigma, h \Rightarrow$  -- only the first arm answers  
    unfold infer at h  
    split at h <;> try (cases h)  
    rename_i  $\rho \tau \_ hP hQ$   
    split at h <;> cases h -- and only if the test passed  
    subst_vars  
    exact .app (infer_sound hP) (infer_sound hQ)
```

Completeness, by rule induction

Lemma

If $\Gamma \vdash M : \sigma$, then $\text{infer } \Gamma M = \text{some } \sigma$.

Proof. A derivation is a finite tree of rule instances, so it suffices to check the claim for each rule, assuming it for the premisses: **rule induction**, one case per rule.

last rule	the induction hypotheses say	so infer computes
(var)	the side condition: $\Gamma.\text{lookup } x = \text{some } \sigma$	$\Gamma.\text{lookup } x$, which is $\text{some } \sigma$
(appl)	$\text{infer } \Gamma M = \text{some } (\rho \rightarrow \sigma)$ and $\text{infer } \Gamma N = \text{some } \rho$	first arm of the match, test $\rho = \rho$ passes: $\text{some } \sigma$
(abst)	$\text{infer } (\Gamma, x:\rho) M = \text{some } \tau$	prefixed with $\rho \rightarrow$: $\text{some } (\rho \rightarrow \tau)$

□ N&G §2.10

Completeness, in Lean

induction d on the derivation produces the three cases of the table; each closes by unfolding `infer` and using the hypotheses.

```
theorem infer_complete { $\Gamma$  M  $\sigma$ } (d :  $\Gamma \vdash M : \sigma$ ) :  
  infer  $\Gamma$  M = some  $\sigma$  := by  
  induction d with  
  | var h          => simpa [infer] using h      -- the side condition  
  | app _ _ ihM ihN => simp [infer, ihM, ihN]    -- the input types agree  
  | abs _ ih       => simp [infer, ih]          -- prefix  $\rho \Rightarrow$ 
```

Decidability, as a tactic

```
-- run 'infer'; the two lemmas turn its answer into a verdict
instance (Γ : Ctx) (M : TTerm) (σ : Ty) : Decidable (Γ ⊢ M : σ) :=
  if h : infer Γ M = some σ then .isTrue (infer_sound h)
  else .isFalse fun d => h (infer_complete d)

example : [] ⊢ ex246 : ((α ⇒ β) ⇒ α ⇒ β) := by decide
example : ¬ ([] ⊢ .app ex246 ex246 : α) := by decide
```

decide evaluates infer. If the result is some σ , soundness supplies the derivation; otherwise completeness shows that none exists. Checking a typing is a computation, not a search, and a refusal is as reliable as an acceptance.

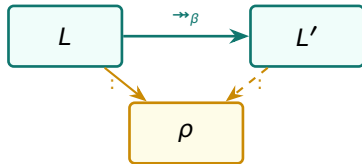
Reduction keeps the type

β -reduction on Λ_T is as on Λ ; only the basis clause gains a type annotation:
 $(\lambda x:\sigma. M) N \rightarrow_\beta M[x := N]$.

Subject Reduction

If $\Gamma \vdash L : \rho$ and $L \rightarrow_\beta L'$, then $\Gamma \vdash L' : \rho$.

Calculation does not change the type: $3 + 5$ is a natural number, and so is 8. Lean's `rfl` and `decide` rely on this.



N&G §2.11, Lem. 2.11.5

Every legal term terminates

■ Theorem (Strong Normalisation, or Termination)

Every legal M is strongly normalising: there is no infinite reduction path starting from M .

The proof uses a measure on legal terms that is positive and decreases at every β -step. In L1, Ω had no normal form; here every term has one.

■ What this does not say

Strong normalisation guarantees termination within a finite number of steps, but sets no bound on that number: “one does not know beforehand how long this waiting will take.”

N&G Thm. 2.11.6, Rem. 2.11.7

$\lambda \rightarrow$ is too weak

Three things Lean says every day, none of them a $\lambda \rightarrow$ type:

```
def id' { $\alpha$  : Type} (x :  $\alpha$ ) :  $\alpha$  := x      -- one identity, for every type
#check (List : Type  $\rightarrow$  Type)              -- a type built from a type
#check (Fin 3 : Type)                        -- a type mentioning a term
```

missing in $\lambda \rightarrow$	example	added by
a term abstracted over a type	id' works at every α	$\lambda 2$ (ch. 3)
a type abstracted over a type	List takes a type to a type	$\lambda \omega$ (ch. 4)
a type depending on a term	Fin 3 mentions the number 3	λP (ch. 5)

Next lecture

Three independent gaps, so three axes. All three together give λC , the type theory of Lean's kernel.

N&G §2.13

References I

Nederpelt, R. and H. Geuvers (2014a). “Simply typed lambda calculus”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 2, pp. 33–68.

— (2014b). *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press.

Chapter 2 of the book throughout: §2.2 simple types (Def. 2.2.1, Notation 2.2.2, Rem. 2.2.4, 2.2.5, 2.2.8, Ex. 2.2.6), §2.3 Church versus Curry typing (Ex. 2.3.1, 2.3.2), §2.4 the derivation system (Def. 2.4.1, 2.4.2, 2.4.5, 2.4.10, Ex. 2.4.6, Rem. 2.4.7, Ex. 2.4.9), §2.5 tree, linear and flag formats, §2.6 the three problems, §2.7–2.9 worked examples and Rem. 2.9.1, §2.10 metatheory (Lem. 2.10.3, 2.10.5, 2.10.7, 2.10.8, 2.10.9, Thm. 2.10.10), §2.11 reduction (Lem. 2.11.1, Thm. 2.11.3, Cor. 2.11.4, Lem. 2.11.5, Thm. 2.11.6, Rem. 2.11.7), §2.12 consequences, §2.13 the drawback, §2.14 the history. Term Finding beyond $\lambda \rightarrow$: §6.3, Thm. 6.3.15.